

Continuous Retrieval-Grounded Reward Design for Secure Code Generation on Small Language Models (CARGO)

Suryansh Singh Sijwali
Pennsylvania State University
University Park, PA, USA
sss6371@psu.edu

Medhansh Kumar Singla
Pennsylvania State University
University Park, PA, USA
mks6942@psu.edu

Suman Saha
Pennsylvania State University
University Park, PA, USA
sumsaha@psu.edu

Abstract—Reinforcement learning (RL) with static application security testing (SAST) feedback is a natural fit for secure code generation. On small language models, this reward is structurally sparse, because most early rollouts fail to compile and all receive the same near-zero reward. In group-relative and ranking-based RL, that homogeneity collapses within-group variation and suppresses useful updates. On Qwen2.5-Coder-1.5B, SAST-only group-relative policy optimization (GRPO) produces a nonzero policy gradient on fewer than 9% of training steps. We propose CARGO (Continuous Augmented Retrieval-Grounded Objective), a reward-design recipe built around a continuous retrieval-grounded reward. This reward, R_{RAG} , scores policy outputs against retrieved secure-fix evidence, which restores gradient flow without replacing the SAST objective. CARGO pairs it with CWE-aware per-prompt reweighting and a σ -floor on the group-relative normalizer. We evaluate CARGO on 1,582 prompts covering 19 common weakness enumerations (CWEs) in Python, C, and C++. Over a SAST-only baseline, it improves Compile@1 by +19.9 pp, Secure@1|Compile by +16.6 pp, and Functional-Secure@1 by +26.5 pp, and over the untrained base policy by +4.4, +5.2, and +13.3 pp. A 4×2 factorial and ablations identify R_{RAG} as the load-bearing component, and the gains reproduce on two 3B models.

Index Terms—secure code generation, reinforcement learning, reward shaping, group-relative policy optimization (GRPO), retrieval-augmented reward, static analysis, small language models, common weakness enumeration (CWE)

I. INTRODUCTION

Code assistants are now embedded in editors, pre-commit hooks, and continuous integration pipelines. The deployment target in those settings is a small language model (SLM), typically in the 1 to 3 billion parameter range, because latency and on-device feasibility rule out larger models. The security properties of code produced by these assistants are mixed. Pearce et al. [1] report that a substantial fraction of Copilot completions on security-sensitive prompts contain Common Weakness Enumeration (CWE) issues. Perry et al. [2] find that users assisted by an AI write more insecure code than users working alone. These findings argue for training-time interventions rather than purely post-hoc filtering [3].

The natural training signal for security is the verdict of a static application security testing (SAST) tool. SAST verdicts are production-grade, language-aware, and free of the

proxy-classifier biases that affect learned reward models. The complication is that SAST verdicts are structurally sparse. An analyzer returns no findings for both secure and non-parsing code, and on a small base policy almost every early rollout fails to parse, so nearly every rollout in a group scores the same near-zero reward. This sparsity is less harmful for value-baselined policy optimization, which does not need within-group variance to produce a gradient. Group-relative and ranking-based variants do need that variance, and they stall under near-constant rewards, a failure mode we characterize and measure in Section II.

We propose **CARGO** (Continuous Augmented Retrieval-Grounded Objective), a reward-design recipe for sparse-reward reinforcement learning (RL) on small language models that is not tied to a single optimizer. The recipe has three components. The first is a continuous retrieval-grounded reward R_{RAG} formed as a cosine similarity between the policy completion and a secure-fix exemplar retrieved from a training-time corpus, with an explicit copy guard. The second is CWE-aware per-prompt gradient reweighting that compensates for supply imbalance across weaknesses, and the third is a σ -floor on the group-relative normalizer, which bounds the low-variance bias that Dr. GRPO [4] identifies. The three components are complementary, since R_{RAG} makes low-variance groups much less common, the σ -floor guards against the residual ones, and CWE reweighting prevents the policy from optimizing only the well-supplied weaknesses. The main configuration runs atop GRPO [5], and a 4×2 factorial repeats it on proximal policy optimization (PPO), REINFORCE leave-one-out (RLOO), and reward-ranked fine-tuning (RAFT).

We evaluate CARGO on a 1,582-prompt benchmark spanning 19 CWEs across Python, C, and C++, held out from training at the prompt level (Section V). We compare CARGO against SAST-only versions of the four RL algorithms, against security-trained releases (SVEN [6], SecCoder-X [7]), and against the base policy with and without inference-time retrieval. CARGO improves Compile@1 by +19.9 percentage points, Secure@1|Compile by +16.6 pp, and Functional-Secure@1 by +26.5 pp over the SAST-only baseline, and the factorial shows that adding R_{RAG} improves every one of the

four algorithms (Section VI).

Our contributions are threefold. First, we formalize the static-analyzer reward-collapse failure mode that affects group-relative and ranking-based RL at SLM scale and measure it on a 1.5B base policy. Second, we introduce CARGO, a three-component reward-design recipe whose central component is the continuous retrieval-grounded reward R_{RAG} . Third, we evaluate it with a 4×2 algorithm-by-retrieval factorial, component ablations, a held-out CWE transfer test, reproductions on Qwen2.5-Coder-3B and StarCoder2-3B, and an offline comparison of retrieval-reward formulations. The ablations include random and adversarial retrieval controls, a σ -floor sweep, and CWE-reweighting and supervised fine-tuning (SFT) warm-start contrasts. We also report that SFT on the same training pool strictly degrades the base policy on all three metrics at 1.5B scale, motivating Kullback-Leibler (KL)-anchored RL rather than further SFT. Our code is available at <https://github.com/SuryanshSS1011/SecureCodeRL-RAG>.

II. MOTIVATION: SAST-REWARD COLLAPSE IN GROUP-RELATIVE RL

This section explains why a SAST-only reward stalls group-relative RL on small models and measures how often it happens. We use GRPO as the running example, since it is the most widely adopted group-relative algorithm, and the measurement shows that the pathology applies equally to RLOO and RAFT.

A. The Group-Relative Update

GRPO [5] samples a group of G rollouts $\{y_i\}_{i=1}^G$ for each prompt, computes per-rollout scalar rewards $r_i \in \mathbb{R}$, and forms a normalized advantage

$$A_i = (r_i - \bar{r}) / (\sigma_r + \epsilon), \quad (1)$$

$$\bar{r} = \frac{1}{G} \sum_i r_i, \quad \sigma_r = \text{std}(\{r_i\}),$$

which replaces the learned value baseline used by PPO [8]. The policy update applies a clipped importance-weighted gradient with A_i as the advantage. When $\sigma_r \rightarrow 0$ the normalizer in Eq. (1) dominates the numerator and every A_i collapses toward zero. The policy gradient, which is proportional to A_i , vanishes accordingly. RLOO replaces $\bar{r}$ with a leave-one-out mean and skips the σ_r division, but its advantage estimate is still bounded by the within-group reward spread. RAFT selects the top- k rollouts as supervised targets and degenerates to random selection when the reward is constant.

B. Why SAST Rewards Collapse Group Variance

Let r_i^{sec} be the SAST-derived security reward for rollout i . An analyzer reports no findings in two different situations, when the rollout is clean and when it does not parse. Our reward aggregator separates the two by parse status, giving clean code 1 and non-parsing code 0 (Table I), so the collapse does not come from confusing them. It comes from the population of rollouts. On a small base policy most early rollouts do not parse, so nearly every rollout in a group receives the same zero reward. The within-group reward distribution is therefore

concentrated at a single value with high probability. This mass-at-a-point distribution has zero variance, so $\sigma_r \rightarrow 0$ and the GRPO update is suppressed. The pathology is not a numerical edge case but the typical condition during early training, when the policy has not yet learned to compile.

C. Empirical Measurement

We instrumented the trainer to log three signals per training step, which are the within-group reward standard deviation σ_r , the per-tool SAST crash rate, and a binary indicator for whether the policy loss exceeds 10^{-6} in absolute value. We then ran a nine-configuration preliminary sweep on Qwen2.5-Coder-1.5B with $G = 16$, no retrieval reward, and no CWE reweighting. The configurations vary the algorithm (GRPO, RLOO, PPO without a value head, RAFT), the gradient weighting (uniform versus CWE-reweighted), and the warm-start condition (with and without SFT). All configurations trained for 1000 steps.

We define the zero-variance-group (zvz) rate as the fraction of steps with $\sigma_r < 0.01$. The nonzero-loss rate is the fraction of steps with $|\mathcal{L}_\pi| > 10^{-6}$, the directly observable signal that the optimizer is updating the policy. Across all nine SAST-only configurations the failure signature is universal. Six GRPO variants (spanning reweight/uniform, $\pm$ SFT warm start, and $\pm$ design-pair inclusion) cluster at $\text{zvz} = 0.91 \pm 0.01$ with nonzero-loss rates of $7.5\% \pm 1.2$ pp. RLOO fares no better, at a nonzero-loss rate of 6.7%, because its leave-one-out advantage is still bounded by the collapsed within-group spread. PPO without a value head and RAFT produce nonzero gradients on 86% and 100% of steps, yet neither improves. PPO’s policy loss is uniformly negative and pushes every completion downward, and RAFT self-distills on an arbitrary top- k because its reward is constant. The main factorial of Section VI instead uses value-baselined PPO. Augmenting GRPO with the retrieval-grounded reward R_{RAG} drops zvz to 0.05 and lifts the nonzero-loss rate to 94.8%, the only configuration in the sweep that trains.

D. Implications for the Approach

The measurement narrows the design space. A learned value head lets PPO sidestep the within-group collapse, but it must fit a value function on the same sparse reward and converges below CARGO, so it is not a complete remedy. An optimizer-only fix, such as removing the standard-deviation division as Dr. GRPO [4] does or flooring it, avoids amplifying near-zero variance but does not change the reward distribution. The fix must therefore create variance at the reward source. A floor can then stabilize the few low-variance groups that remain, and Section IV presents the CARGO formulation that combines the two.

III. RELATED WORK

A. Reinforcement Learning for Secure Code Generation

SVEN [6] fine-tunes a code model on a curated set of vulnerable-versus-fixed pairs using prefix tuning, and reports

non-trivial security improvements on a held-out CWE evaluation. SafeCoder [9] extends the idea to instruction-style fine-tuning. Both treat security as a supervised-fine-tuning objective rather than as a reinforcement signal, and neither addresses the GRPO-specific reward-collapse failure mode that CARGO targets. A separate family of systems uses a learned classifier as a reward proxy under PPO. The closest is SecCoder-X [7], which synthesizes vulnerability-inducing tasks, trains a reasoning-based vulnerability reward model, and then optimizes a binary vulnerability reward in an on-line RL loop. CARGO differs on several axes. Its reward comes directly from production-grade static analyzers, namely CodeQL [10], Semgrep [11], and Bandit [12], rather than from a learned proxy. Its security reward is also continuous through R_{RAG} rather than binary, which restores within-group variance at the 1.5B to 3B scale we target. CARGO preserves functionality through the reliability reward and the stub penalty rather than a reward model. Its per-step cost is dominated by analyzer calls rather than reward-model inference, and its failure mode is analyzer recall, since a vulnerability the tools miss goes unpenalized.

Functional-correctness RL on code is a parallel literature. CodeRL [13] uses unit-test rewards under an actor-critic objective, and StepCoder [14] extends this to compiler feedback. In prior work [15] we demonstrate partial-credit functional reward shaping for SLM code generation and identify CWE-mapped security rewards as an open direction. CARGO takes that direction on the security side, but with a different shaping mechanism. Rather than staging the SAST reward into discrete tiers, we add a continuous retrieval-grounded reward that operates alongside the SAST verdict.

B. Group-Relative Policy Optimization

GRPO originates in DeepSeekMath [5], and Dr. GRPO [4] later showed that dividing the advantage by the group standard deviation over-weights questions whose rewards barely vary, so it removes that division. CARGO instead keeps the normalizer and floors it (Section IV-C). Open-RS [16] reports that GRPO is viable on a 1.5B base policy but observes optimization instability past the first few hundred steps, the regime in which CARGO operates.

C. Retrieval-Augmented Code Generation

ReACC [17] and CodeRAG-Bench [18] use retrieval at *inference* time to ground completions in similar context. CARGO uses retrieval at *training* time, as a component of the reward signal. Inference-time retrieval biases what the policy outputs at generation time, whereas reward-time retrieval shapes what the policy learns, so the secure pattern is internalized in the weights rather than supplied as a runtime hint. The closest prior use of retrieval for security is VulRAG [19], which retrieves vulnerability knowledge to aid *detection*, whereas CARGO folds the retrieved secure exemplar into the RL *reward*. Section VI includes an inference-time retrieval control.

IV. APPROACH: CARGO

CARGO augments a sparse, static-analyzer-derived security reward with three components that restore gradient flow on small language models. The main configuration instantiates the recipe on GRPO, the most memory-efficient group-relative algorithm at the 1.5B base policy we target.

A. Continuous Retrieval-Grounded Reward R_{RAG}

Let x denote a training prompt and y a policy completion. Each training prompt is associated with a per-CWE pool of *secure exemplar* snippets, each derived from a published vulnerability fix or an authored design pair. Concretely, exemplars are drawn from CVEfixes [20] and DiverseVul [21] fix-side commits, from Juliet [22] known-good test cases, and from authored design pairs for the under-represented authorization and authentication CWEs (CWE-306, CWE-862). We index this pool by the prompt and the exemplar text using a hybrid BM25-plus-dense retriever, whose dense side uses bge-base embeddings [23] and whose sparse side uses the BM25S library. At training time the retriever returns the top-ranked secure exemplar $e^+(x)$ for the prompt’s target CWE. On a retrieval miss, when nothing matches the CWE or its parent, the term is set to zero and the reward composes with R_{sec} and R_{rel} alone.

The reward is a *retrieval-grounded cosine* between the policy completion and the retrieved secure exemplar,

$$R_{\text{RAG}}(y, e^+) = \cos(\phi(y), \phi(e^+(x))) \cdot \mathbb{1}[\cos(\phi(y), \phi(e^+)) \leq \tau_{\text{copy}}], \quad (2)$$

where ϕ is the bge-base encoder used for the retrieval index. The threshold $\tau_{\text{copy}} = 0.95$ is an explicit copy guard that zeroes R_{RAG} when the completion is a verbatim or near-verbatim copy of the exemplar. The reward is continuous-valued in $[-1, \tau_{\text{copy}}]$.

The role of R_{RAG} is deliberately *not* to encode the security decision, which is the job of R_{sec} . It is a dense, continuous, retrieval-grounded prior that restores within-group variance when R_{sec} collapses. Three properties matter for the group-relative regime of Section II. First, it is bounded and continuous in the policy output, so any update that moves y in embedding space changes the reward smoothly, unlike the discrete SAST verdict that flips only at finding thresholds. Second, $\phi(e^+)$ is retrieval-grounded and varies per prompt, so the same emitted code scores differently across prompts and the policy cannot collapse to a single verbatim attractor. Third, the copy guard prevents the degenerate solution of memorizing top-ranked exemplars. Because the secure decision stays with R_{sec} , R_{RAG} acts as a dense shaping signal that speeds the policy toward the R_{sec} -defined optimum rather than relocating it.

Section VI reports three controls. A *random retrieval* control replaces the top-ranked exemplar with a uniformly sampled one (any CWE), testing whether CWE-conditioned retrieval matters. An *adversarial retrieval* control retrieves the lowest-ranked exemplar within the prompt’s CWE, testing whether retrieval rank matters. A *binary* control replaces Eq. (2) with

the indicator $\mathbb{1}[\cos(\phi(y), \phi(e^+)) > \cos(\phi(y), \phi(e^-))]$, where e^- is the pre-fix, vulnerable version of the retrieved exemplar, testing whether a binary signal that also consults the vulnerable version can replace the continuous reward.

a) *On the choice of ϕ :* The encoder ϕ (bge-base) is a general-purpose text embedder applied to code as text and is not trained for security. What R_{RAG} requires of ϕ is weaker, namely a dense signal that separates CWE-matched secure exemplars from unrelated code well enough to break reward homogeneity. The retrieval controls test this directly, and CWE-matched retrieval beats both random and within-CWE adversarial controls (Section VI-C), so the embedding supplies a task-conditioned prior that restores within-group variance rather than a security verdict, as Section VI-F quantifies. The signal is weakest where vulnerable and fixed code are near-identical in surface form, as in the missing-authorization and missing-authentication CWEs whose fix is a single added check. That weakness is why R_{RAG} composes with the SAST verdict rather than replacing it.

B. CWE-Aware Per-Prompt Gradient Reweighting

The training pool is imbalanced across the trained CWEs by roughly two orders of magnitude. A uniform pass over the pool would spend most of the gradient budget on well-supplied weaknesses such as CWE-787 buffer overflow, CWE-79 XSS, and CWE-89 SQL injection. It would effectively skip supply-poor categories such as CWE-862 missing authorization and CWE-306 missing authentication.

We apply a per-prompt scalar weight $w(x) > 0$ to the per-prompt loss contribution

$$w(x) = \frac{(|C|/n_{c(x)})^{\alpha_{\text{cwe}}}}{Z}, \quad (3)$$

where $c(x)$ is the CWE label of prompt x , $|C|$ is the number of trained CWEs, and n_c is the number of training prompts labeled c . The exponent $\alpha_{\text{cwe}} \in [0, 1]$ controls the strength of the reweighting, and Z is a normalizer chosen so that $\mathbb{E}_x[w(x)] = 1$ over a sampled batch. We use $\alpha_{\text{cwe}} = 0.5$. The exponent dampens the raw inverse-frequency reweighting to avoid over-amplifying the smallest categories. The reweighting scales the per-prompt expected gradient magnitude without changing within-group advantage normalization, so it composes with each group-relative algorithm in our factorial, and for PPO it scales the per-prompt clipped surrogate loss equivalently.

C. σ -Floor on the Group-Relative Normalizer

To bound the low-variance bias that Dr. GRPO [4] identifies without removing the normalizer, we replace the denominator of Eq. (1) with $\max(\sigma_r, \sigma_{\min})$, where $\sigma_{\min}$ is a positive lower bound. We use $\sigma_{\min} = 0.05$ in the main configuration. The floor does not eliminate the zero-variance failure mode by itself, but it does prevent the $1/\sigma_r$ term from amplifying noise on the residual low-variance groups that survive after R_{RAG} is added. Section VI-C sweeps $\sigma_{\min}$, and its $\sigma_{\min} = 0$ setting tests whether the floor adds value when R_{RAG} is present.

TABLE I
REWARD BY ROLLOUT STATE.

Rollout state	R_{rel}	R_{sec}	r
Does not parse	0.00	0.00	0.00
Compiles, analyzer-clean, no run	0.33	1.00	0.53
Compiles, two high-severity findings	0.33	0.00	0.23
Runs with output, one finding	1.00	0.30	0.79
Runs with output, analyzer-clean	1.00	1.00	1.00

D. Combined Training Objective

For each prompt x and each rollout y_i in the group of G samples, the scalar reward is the additive combination

$$r_i = \alpha_{\text{mix}} R_{\text{sec}}(y_i) + (1 - \alpha_{\text{mix}}) R_{\text{rel}}(y_i) + \lambda_{\text{rag}} R_{\text{RAG}}(y_i, e^+) - \beta \mathbb{1}[y_i \text{ is a stub}], \quad (4)$$

with $\alpha_{\text{mix}} = 0.3$, $\lambda_{\text{rag}} = 0.1$, and $\beta = 1.5$.

Each component plays a distinct role. $R_{\text{sec}}(y_i) \in [0, 1]$ is the SAST-derived security score. It aggregates findings from CodeQL [10] and Semgrep [11], from Bandit [12] for Python, and from Cppcheck [24] plus GCC for C and C++, and it weights them by common vulnerability scoring system (CVSS) severity¹ and assigns 0 to non-parsing rollouts. $R_{\text{rel}}(y_i) \in [0, 1]$ is a staged reliability reward following our prior work [15], with graded credit for parsing, compiling, running without crashing, and producing the expected output. R_{RAG} is the retrieval-grounded cosine of Eq. (2), with copy guard. The $\beta \mathbb{1}[\cdot]$ term is an anti-Goodhart stub penalty that subtracts a fixed amount when a detector flags a trivial return-only or empty-body stub. Without it, an early-training policy converges to stubs because empty completions satisfy the no-finding state of SAST tools, routing the variance-collapse failure through the output distribution rather than the reward distribution.

We combine additively rather than multiplicatively because a product such as $R_{\text{sec}} \cdot R_{\text{RAG}}$ would zero the shaping gradient when R_{sec} is uniformly zero early in training, reintroducing the variance collapse R_{RAG} is meant to prevent. The choice $\alpha_{\text{mix}} = 0.3$ keeps the reliability backbone dominant early in training, when SAST verdicts on SLM-emitted code are sparser than per-test functional verdicts. A small nonzero security weight still lets R_{sec} activate once compile rates rise. We do not sweep α_{mix} , because the contribution under study is R_{RAG} , and the $\alpha_{\text{mix}} = 0$ ablation in Section VI-C exercises the security-versus-reliability axis.

Table I maps representative rollout states to R_{rel} , R_{sec} , and the final reward for a prompt without unit tests, where compiling, running, and producing output each earn one third of R_{rel} . The aggregator separates non-parsing from analyzer-clean code by parse status, so the two are not reward-equivalent. A non-parsing rollout instead shares its zero security reward with a rollout that carries two or more high-severity findings. The homogeneity that collapses a group therefore comes from early training, when almost every rollout fails to parse and scores

¹<https://www.first.org/cvss/>

Algorithm 1 CARGO training step

- 1: Sample prompt x uniformly from the training pool and look up $w(x)$
 - 2: Retrieve top-ranked secure exemplar $e^+(x)$ from the retrieval index
 - 3: Sample G rollouts $\{y_i\}$ from current policy $\pi_\theta(\cdot | x)$
 - 4: Compute $R_{\text{sec}}(y_i)$ via an aggregated SAST check
 - 5: Compute $R_{\text{rel}}(y_i)$ via staged parse / compile / run / output check
 - 6: Compute $R_{\text{RAG}}(y_i, e^+)$ via Eq. (2)
 - 7: Form r_i via Eq. (4)
 - 8: Compute $\bar{r}$, σ_r over group
 - 9: Form $A_i = (r_i - \bar{r}) / \max(\sigma_r, \sigma_{\min})$
 - 10: Compute clipped GRPO surrogate loss $\mathcal{L}_{\text{GRPO}}$ with advantages $\{A_i\}$
 - 11: Take optimizer step on $w(x) \cdot \nabla_\theta \mathcal{L}_{\text{GRPO}}$
-

near zero, rather than from one value shared by secure and insecure code.

The CARGO step is then

$$\nabla_\theta \mathcal{L}_{\text{CARGO}} = \mathbb{E}_x \left[w(x) \cdot \nabla_\theta \mathcal{L}_{\text{GRPO}}(\{y_i\}, \{r_i\}, \sigma_{\min}) \right] \quad (5)$$

where $\mathcal{L}_{\text{GRPO}}$ is the standard clipped surrogate loss with the σ -floored advantage from Section IV-C, $\{r_i\}$ is the combined reward of Eq. (4), and $w(x)$ is the per-prompt weight of Eq. (3).

Algorithm 1 gives the per-step procedure, in which three additive reward sources feed a CWE-weighted, σ -floored group-relative update.

V. EVALUATION

A. Benchmark

We assemble a 1,582-prompt evaluation set by aggregating CWE-aligned prompts from six sources covering 19 CWEs across Python, C, and C++. The per-source composition is Juliet 1.3 [22] 872, CyberSecEval [25] 336, SecCodePLT [26] 241, CASTLE [27] 90, SecurityEval [28] 23, and CWEval [29] 20. By language, C accounts for 564 prompts, Python 557, and C++ 461. Juliet 1.3 dominates the C/C++ memory-safety classes, while the Python-heavy SecCodePLT and CyberSecEval sources provide the injection and authorization classes.

The evaluation pool is prompt-level disjoint from the training pool. Training draws on CVEfixes [20], DiverseVul [21], and a separate Juliet 1.3 [22] extraction, while the six sources supply the test prompts. Juliet contributes to both pools through disjoint extractions, so we audit the separation. A near-duplicate audit at four cosine thresholds $\{0.3, 0.5, 0.7, 0.9\}$ between every training prompt and every evaluation prompt finds a maximum best-match Levenshtein ratio of 0.43, and zero pairs exceed any threshold at or above 0.5. Three of the 19 evaluation CWEs (CWE-326 inadequate encryption, CWE-328 weak hash, CWE-798 hard-coded credential) appear in evaluation only and not in training. We report these as a held-out CWE transfer test in Section VI, treating CWE-326 cautiously because it has only $n = 4$ prompts.

B. Metrics

We report three primary metrics. **Compile@1** (Cmp@1) is the fraction of single-sample greedy completions that parse and, for C and C++, pass a compile-only check, over all 1,582 prompts. **Secure@1|Compile** (Sec|Cmp) is the fraction of compile-passing completions that pass an aggregated SAST check with no high-severity findings, so its denominator is each system’s compile count. **Functional-Secure@1** (Func-Sec@1) is the fraction of single-sample completions that pass their unit tests and SAST on the 156-prompt test-equipped subset, where compile or run failures contribute zero to the numerator. The denominator stays at 156 for every system so the column is comparable. A system that fails to compile a test-equipped prompt counts as a non-pass on that row rather than leaving the denominator. Because unit tests exist only for this subset, Functional-Secure@1 is a joint correctness-and-security signal on the smaller test-equipped subset rather than over all 1,582 prompts. The subset is Python only and comes from SecCodePLT, the Python half of CWEval, and the authored CWE-306 and CWE-862 prompts, so Functional-Secure@1 does not cover the C and C++ memory-safety CWEs.

Two secondary metrics describe training dynamics rather than the final policy. These are the **zero-variance-group rate** and the **nonzero-loss rate** defined in Section II-C, which we reuse here to show that CARGO trains where the SAST-only baselines stall.

We sized the comparisons for paired McNemar tests on prompt-level pass indicators, with a minimum detectable effect (MDE) of 15 percentage points fixed before evaluation. Confidence intervals are Wilson 95-percent. The gains over the SAST-only baseline and every gain from adding R_{RAG} in the factorial exceed this threshold. The gains over the base policy and the component differences in Section VI-C fall below it, so we report them as observed differences rather than detected effects.

C. Baselines

a) Base, retrieval, and SFT baselines: We evaluate the base policy Qwen2.5-Coder-1.5B-Instruct [30] greedily, the same policy with inference-time retrieval that prepends the top- k exemplar to the prompt, and an SFT-only fine-tune of the base policy on CARGO’s training pool.

b) Security-trained releases: We compare against the SVEN release [6] on a CodeGen-2.7B base and two SecCoder-X checkpoints [7] on Qwen2.5-Coder-3B and -7B bases. We omit SafeCoder [9] from the table because the released checkpoint emits the end-of-sequence token on the first decoded position for every prompt in our benchmark, producing empty completions that are uniformly counted as non-compiling. We re-tested SafeCoder with multiple prompt formats and decoding configurations, and the failure is independent of those choices. We treat this as a release-side incompatibility rather than evidence about the SafeCoder method.

c) *SAST-only RL ablations*: We run a 4×2 factorial of algorithms by retrieval condition. The algorithms are GRPO [5], value-baselined PPO [8], RLOO [31], and RAFT [32]. The retrieval conditions are SAST-only (no R_{RAG}) and the combined reward of Eq. (4). The SAST-only condition retains the reliability reward, the stub penalty, the CWE reweighting, and the σ -floor, and removes only R_{RAG} . This factorial tests whether CARGO’s gains come from the algorithm choice, from the reward augmentation, or from their interaction.

D. Training Protocol

We implement CARGO with PyTorch and the HuggingFace Transformers and PEFT libraries.² Unless otherwise noted, RL configurations train for 1000 steps on Qwen2.5-Coder-1.5B-Instruct with rank-16 low-rank adaptation (LoRA) adapters [33] on the attention projections. They use group size $G = 16$, batch size 4 prompts, AdamW with cosine decay, and a peak learning rate of 1×10^{-5} . The reward weights and the σ -floor take the values given in Section IV, and the KL coefficient is $\beta_{\text{KL}} = 0.05$ against the frozen base policy. We did not tune these on the evaluation set. They were fixed before evaluation from short pilot runs and standard GRPO values at this scale, and Section VI-C varies the components rather than searching them. The retrieval index is built once per corpus version and reused across retrieval-enabled configurations. Analyzer invocations run in a sandbox with a 60-second per-tool timeout, except for CodeQL on C and C++ which uses a 120-second timeout.

Five configurations use $G = 8$ rather than $G = 16$ for memory reasons. The two PPO runs and the SFT warm-start run carry a value head or a warm-start adapter alongside the policy, reference policy, and retriever. At $G = 16$ their activations exceed the 40 GB of the A100 training GPU. The two cross-family runs use 3B base policies (Qwen2.5-Coder-3B and StarCoder2-3B) whose larger parameter count likewise overflows at $G = 16$. We expect these comparisons to transfer across group sizes, because the 22.5 to 26.5 pp gains from adding R_{RAG} far exceed any effect we would expect from halving the group. We did not measure a $G = 8$ GRPO control. SFT warm-start runs precede the RL run with 20000 steps of supervised fine-tuning on the training pool, using the same LoRA rank and schedule and selecting the checkpoint with the lowest validation loss as the warm-start adapter. Evaluation uses greedy decoding (temperature 0) with at most 512 new tokens.

E. Compute

Experiments run on single NVIDIA A100 40 GB and A40 45 GB GPUs and consume 683 GPU-hours in total across the 22 training runs, the 21 evaluation passes, corpus rebuild, indexing, and training-side analysis.

²<https://pytorch.org>, <https://github.com/huggingface/transformers>, <https://github.com/huggingface/peft>

TABLE II
CARGO VERSUS BASELINES.

System	Cmp@1	Sec Cmp	Func-Sec
<i>Untrained baselines</i>			
Qwen2.5-Coder-1.5B (base)	55.7	90.2	21.8
Qwen2.5-Coder-3B	56.1	89.3	20.5
Qwen2.5-Coder-7B	56.2	89.8	23.1
StarCoder2-3B [34]	26.9	93.7	7.1
<i>Security-trained releases</i>			
SVEN [6] (CodeGen-2.7B)	35.3	98.4	7.7
SecCoder-X [7] (Qwen2.5-Coder-3B)	56.1	92.2	16.0
SecCoder-X [7] (Qwen2.5-Coder-7B)	56.4	93.5	23.1
<i>Trained on our corpus</i>			
SFT-only (full fine-tune)	50.5	85.4	14.7
GRPO + SAST only	40.2	78.8	8.6
CARGO	60.1	95.4	35.1

VI. RESULTS

A. Main Results

Table II compares CARGO with every baseline. Over the SAST-only baseline, CARGO improves Compile@1 by +19.9 pp, Secure@1|Compile by +16.6 pp, and Functional-Secure@1 by +26.5 pp on the test-equipped subset. Against the untrained base policy the same metrics improve by +4.4, +5.2, and +13.3 pp, so CARGO exceeds both the SAST-only baseline and the base model. The gain over SFT-only isolates what RL adds beyond supervised fine-tuning on the same corpus.

Across seeds {42, 1337, 2024} Func-Sec@1 is 35.1 ± 0.4 , so seed variation is far smaller than the gains. The 95% Wilson intervals for Func-Sec@1 are [28.1, 42.9] for CARGO, [16.0, 28.9] for the base policy, and [5.1, 14.1] for the SAST-only baseline.

a) *SFT as a negative baseline*: The SFT-only row of Table II is a negative result. Naive cross-entropy SFT on CARGO’s training pool *strictly degrades* the base policy on all three metrics. Compile@1 drops by 5.2 pp ($55.7 \rightarrow 50.5$), Secure@1|Compile by 4.8 pp ($90.2 \rightarrow 85.4$), and Functional-Secure@1 by 7.1 pp ($21.8 \rightarrow 14.7$). The degradation holds across LoRA ranks $r \in \{16, 32, 64, 128, 256, 512\}$ and full fine-tuning. It is not an under-training artifact, because validation loss improves with capacity while the metrics fall, and it persists on the training-matched Juliet 1.3 subset. We attribute the failure to the SLM regime, where fitting secure-fix substitutions can evict compile competence and teach substitution shape rather than the security property. The + SFT warm start configuration in Table III uses the SFT checkpoint with the lowest validation loss as an RL warm start. It avoids this degradation and suppresses early stub-emission Goodhart failure, cutting the stub rate from about 10–21% to about 1.4%. Thus, SFT is useful as an *initializer* even though it is a worse *terminal* policy.

B. Factorial: Algorithm $\times$ Retrieval

Table IV reports the 4×2 contrast. The + R_{RAG} column exceeds the SAST-only column for every algorithm, but the

TABLE III
COMPONENT ABLATIONS.

Configuration	Cmp@1	Sec Cmp
Full CARGO	60.1	95.4
– R_{RAG} (SAST-only)	40.2	78.8
– CWE reweight	60.2	95.0
– σ -floor	57.5	93.1
+ SFT warm start	60.4	95.7

TABLE IV
ALGORITHM $\times$ RETRIEVAL FACTORIAL (FUNC-SEC@1).

Algorithm	SAST only	+ R_{RAG}	Δ
GRPO	8.6	35.1	+26.5
PPO (with value head)	10.5	36.2	+25.7
RLOO	8.6	33.7	+25.1
RAFT	7.9	30.4	+22.5

mechanism differs by algorithm. GRPO depends on within-group variance, which R_{RAG} supplies, and gains the most at +26.5 pp. PPO’s value head partially compensates for the sparsity, so it scores higher both without and with retrieval, reaching 36.2 against GRPO’s 35.1, while gaining slightly less at +25.7 pp. RLOO also depends on within-group variance and gains +25.1 pp. RAFT operates on top- k ranks rather than continuous advantages, so the continuous nature of R_{RAG} is less directly useful and the uplift is smallest. All four gains lie within 4 pp of one another, so the reward rather than the estimator carries the effect. The 95% Wilson interval of every + R_{RAG} result lies above that of every SAST-only result, with lower bounds of at least 23.7 against upper bounds of at most 16.3.

C. Component Ablations

Table III reports Compile@1 and Secure@1|Compile when each component is removed from the full recipe. The CWE-reweight and SFT warm-start variants lie within seed variation (≤ 0.4 pp) of Full CARGO. Removing the σ -floor costs 2.6 and 2.3 pp, which falls below the 15 pp minimum detectable effect, so the floor acts as a safeguard rather than a load-bearing component, as the sweep below also shows. Removing R_{RAG} costs 19.9 and 16.6 pp, and the training dynamics match Section II-C, since the σ -floor and CWE reweighting alone leave the nonzero-loss rate on the collapse plateau at 0.09 and 0.07. The retrieval term is therefore the load-bearing component for restoring gradient flow.

a) *CWE reweighting and σ -floor*: The CWE-reweighting row of Table III is identifiable only when R_{RAG} is present. Without R_{RAG} , every GRPO configuration sits on the zero-variance plateau of Section II however the per-prompt weights are set, as the SAST-only sweep of Section II-C confirms. A $\sigma_{\min}$ sweep over $\{0, 0.01, 0.05, 0.1\}$ plus an annealed $0.1 \rightarrow 0.01$ schedule is flat once R_{RAG} is present, with all settings within ± 1 pp on Func-Sec@1, and we keep $\sigma_{\min} = 0.05$ to guard residual low-variance groups.

b) *Stub penalty β and security weight α_{mix}* : Removing the stub penalty ($\beta=0$) reproduces the Goodhart attractor. The policy converges to empty stubs that trivially satisfy SAST and scores Cmp@1 27.5, Sec|Cmp 97.6, and Func-Sec 5.3, which is high security on a near-empty output distribution. Removing the security reward ($\alpha_{\text{mix}}=0$), which leaves only R_{rel} and R_{RAG} , yields Cmp@1 58.4, Sec|Cmp 90.0, and Func-Sec 26.3. R_{RAG} alone restores variance and lifts the compile rate, but the security direction must come from R_{sec} . Together these show that each additive term is necessary.

c) *R_{RAG} design*: A 2×2 design crosses the two axes of R_{RAG} , *continuous vs. binary* and *CWE-matched vs. random*. Func-Sec@1 is 35.1 for continuous CWE-matched retrieval, which is CARGO, 25.7 for binary CWE-matched, 21.7 for continuous random, and 15.8 for binary random. CWE-matched retrieval thus beats random retrieval by 13.4 pp, and the adversarial control, which scores 30.3, falls between the two, so retrieval rank within a CWE contributes incrementally. The binary variants use the contrastive indicator of Section IV-A, so they differ from CARGO in both shape and signal. The binary CWE-matched control carries the secure-versus-vulnerable direction that the cosine lacks, yet it scores 9.4 pp lower. Early in training it left 36% of its groups tied, which yields no gradient, and updated on 64% of steps, against 6% and 94% for the continuous reward. This suggests that the continuous reward helps mainly by keeping groups from tying rather than through the direction it encodes. We therefore prefer it over the staged-binary reward of our prior work [15], whose tiers would tie in the same way because SAST findings are categorical rather than strictly ordered. CWE matching and the continuous shape both point in the expected direction, but the 13.4 and 9.4 pp gaps fall below the 15 pp detectable effect, so we report them as observed differences.

d) *Inference-time vs reward-time retrieval*: A natural alternative is *inference-time* retrieval, which prepends the secure exemplar $e^+(x)$ to the prompt at evaluation and leaves the policy untrained. If CARGO’s gain came from the retrieved information itself, both forms of retrieval should help, but if it came from training-time gradient shaping, only reward-time retrieval should. Inference-time prepending yields Compile@1 56.4, Sec|Cmp 91.0, and Func-Sec 24.3, against 60.1, 95.4, and 35.1 for CARGO. It adds at most 2.5 pp to the base policy, which indicates that the training-time gradient is the channel for the security signal.

D. Held-Out CWE Transfer

On the held-out CWEs, Secure@1|Compile rises from the base policy’s 18.5 to CARGO’s **28.7** for CWE-328 ($n=62$) and from 22.9 to **34.4** for CWE-798 ($n=139$). Both exceed the SAST-only GRPO row at 6.8 and 8.1, which indicates that the recipe transfers rather than memorizing per-CWE patterns. But the 95% Wilson intervals at these sample sizes are 14 to 22 pp wide, so the two results are suggestive rather than conclusive. CWE-326 ($n=4$) is too small to report reliably.

TABLE V
CROSS-FAMILY REPRODUCTION (FUNC-SEC@1).

Base policy	Base	+ CARGO	Δ
Qwen2.5-Coder-1.5B (main)	21.8	35.1	+13.3
Qwen2.5-Coder-3B	20.5	41.4	+20.9
StarCoder2-3B	7.1	26.3	+19.2

TABLE VI
OFFLINE COMPARISON OF RETRIEVAL-REWARD VARIANTS.

Variant	Juliet AUC			PrimeVul AUC	Juliet	
	bge	UniX	CodeB		std	Twin
Cosine (CARGO)	0.79	0.83	0.83	0.51	0.058	+0.94
Contrastive	0.94	0.94	0.89	0.50	0.038	-0.06
Projection	0.96	0.95	0.90	0.50	0.101	-0.16
Lexical overlap	0.95	0.96	0.95	0.51	0.297	-0.51

E. Cross-Family Reproduction

We rerun the CARGO recipe on Qwen2.5-Coder-3B [30], a within-family scale-up, and on StarCoder2-3B [34], a model from another family. Table V reports Func-Sec@1 with and without CARGO. CARGO lifts all three base policies, including StarCoder2-3B, which we flagged before evaluation as a likely negative, so its gain is not specific to Qwen2.5-Coder-1.5B. For both 3B models the 95% Wilson intervals of the base and CARGO results do not overlap, whereas for the 1.5B policy they overlap slightly, consistent with its gain falling below the detectable effect.

F. Retrieval-Reward Design Comparison

CARGO’s cosine reward already restores a within-group standard deviation of 0.058, so we asked offline which member of the retrieval-reward family separates secure from vulnerable code best and restores the most variance. We scored eight variants under three encoders, which were bge-base-en-v1.5, UniXcoder [35], and CodeBERT [36], on matched secure and vulnerable pairs from Juliet C, the authored design pairs, and PrimeVul [37] fixes of real Common Vulnerabilities and Exposures (CVE) entries. Table VI reports four representative variants, with the PrimeVul AUC, standard-deviation, and twin columns under bge. The contrastive variant subtracts the similarity to the vulnerable version, $\cos(\phi(y), \phi(e^+)) - \cos(\phi(y), \phi(e^-))$, and the binary control of Section VI-C is its thresholded form. The lexical-overlap variant is embedding-free, and with $\mathcal{A}$ and $\mathcal{D}$ the identifier tokens the fix added and deleted and $T(y)$ the completion’s identifier set, it scores $(|T(y) \cap \mathcal{A}| - |T(y) \cap \mathcal{D}|) / (|\mathcal{A}| + |\mathcal{D}|)$. The fix-direction projection scores a completion by its alignment with the direction $\phi(e^+) - \phi(e^-)$ that the retrieved pair defines. The pre-fix version e^- comes from the same fix commit as e^+ , so the pair needs no corpus beyond the one CARGO already builds.

Three findings hold across encoders and corpora. First, within a CWE family the fix-direction projection separates secure from vulnerable code far better than CARGO’s cosine

reward under every encoder. Its area under the ROC curve (AUC) on Juliet is 0.90 to 0.96, against 0.79 to 0.83 for the cosine, and the embedding-free lexical variant comes within 0.01 of it or exceeds it. Second, the projection carries 1.6 to 15.6 times the within-group standard deviation of the plain contrastive form across encoders and corpora, so it supplies more usable gradient before λ_{rag} scales it. Third, the cosine reward pays the exemplar’s vulnerable twin between +0.84 and +1.00 across encoders, because a one-line fix leaves the two versions near-identical in embedding space. The contrastive and projection forms score that twin at or below zero.

Every variant scores AUC near 0.50 across different functions on PrimeVul, so none transfers a security verdict to a novel function. The retrieval reward is a task-conditioned prior that restores variance and ranks the matched fix above its vulnerable twin, and the security direction comes from R_{sec} .

For a next iteration we would adopt the projection reward and replace the cosine copy guard with an exact and 5-gram Jaccard filter at 0.85. The cosine guard triggers on 44 to 69 percent of legitimate alternative solutions on templated corpora. We would also raise λ_{rag} so that λ_{rag} times the standard deviation of R_{RAG} approaches σ_{min} , which places λ_{rag} near 0.3 to 2 rather than 0.1. Full RL validation of these changes would follow.

VII. LIMITATIONS AND FUTURE WORK

CARGO targets the 1B–3B regime where variance collapse (Section II) is most acute. The benchmark covers Python, C, and C++ across 19 CWEs evaluated greedily at @1. JavaScript, Go, Rust, Java, and PHP are absent, and the training pool inherits the language imbalance of CVEfixes and DiverseVul, which over-represent C relative to web-stack languages. The near-duplicate audit bounds surface similarity between the pools, but Juliet generates many structurally equivalent variants of each flaw, so semantically equivalent cases could still appear in both. Secure@1|Compile is bounded by analyzer recall, since CARGO cannot surface vulnerabilities the underlying SAST tools miss. Because the same analyzers supply the training reward and the evaluation metric, Secure@1|Compile measures security as those analyzers define it. An independent oracle such as a held-out analyzer, dynamic tests, or a manual audit would strengthen the security claim. Post-hoc audits found no verbatim-copy reward hacking against the retrieval index, but rule-level, analyzer-specific evasion remains possible and may score well under our metric. General code-quality regression on HumanEval-style prompts [38] is unmeasured, and the KL anchor limits but does not eliminate drift. The mixing weight λ_{rag} and floor σ_{min} are hand-tuned, and the main result centers on one primary base policy with two 3B reproduction checks.

Four directions follow. First, *characterizing the SLM/LLM crossover* would identify the parameter regime where variance collapse disappears and simpler SAST-only recipes become competitive. Second, *multi-language coverage* would extend the benchmark and training pool to JavaScript and Java, both

covered by Semgrep and CodeQL. Third, *execution-based correctness* would pair CARGO’s security signal, R_{sec} together with R_{RAG} , with a staged functional reward. We did not run this pairing because functional tests exist only for the test-equipped subset. Fourth, *adaptive reward mixing* would schedule λ_{rag} in response to the within-group σ_r of R_{sec} alone, which could improve sample efficiency.

VIII. CONCLUSION

We characterized a failure mode in which group-relative and ranking-based RL stalls on small base policies trained with a static-analyzer-only security reward, because the within-group reward distribution collapses to a point. CARGO restores gradient flow by combining a continuous retrieval-grounded reward R_{RAG} , CWE-aware per-prompt gradient reweighting, and a σ -floor on the group-relative normalizer. On our 1,582-prompt benchmark, it improves Compile@1 by +19.9, Secure@1|Compile by +16.6, and Func-Sec@1 by +26.5 pp over the SAST-only baseline, and by +4.4, +5.2, and +13.3 pp over the base policy. The recipe is not tied to a single optimizer, since adding R_{RAG} improves GRPO, PPO, RLOO, and RAFT alike. CARGO also reproduces on Qwen2.5-Coder-3B and StarCoder2-3B and transfers to two held-out CWEs, with the third too small to assess. Random and binary-thresholded variants of R_{RAG} score lower than CARGO, although only the variant that is both random and binary falls beyond the 15 pp detectable effect.

ACKNOWLEDGMENT

The authors recognize the Penn State Institute for Computational and Data Sciences (ICDS) (RRID:SCR_025154) for providing access to computational research infrastructure (RRID:SCR_026424).

REFERENCES

- [1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” *Communications of the ACM*, vol. 68, no. 2, pp. 96–105, 2025.
- [2] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?” in *CCS*, 2023.
- [3] Meta AI, “CodeShield: A lightweight filter for LLM-generated insecure code,” <https://github.com/meta-llama/PurpleLlama/tree/main/CodeShield>, 2024.
- [4] Z. Liu *et al.*, “Understanding R1-Zero-like training: A critical perspective,” in *Conference on Language Modeling (COLM)*, 2025.
- [5] Z. Shao *et al.*, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [6] J. He and M. Vechev, “Large language models for code: Security hardening and adversarial testing,” in *CCS*, 2023.
- [7] D. Wu, Y. Liu, T. Y. Zhuo, H. Zhang, and S.-K. Ng, “Secure code generation via online reinforcement learning with vulnerability reward model,” *arXiv preprint arXiv:2602.07422*, 2026.
- [8] J. Schulman *et al.*, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] J. He, M. Vero, G. Krasnopolka, and M. Vechev, “Instruction tuning for secure code generation,” in *ICML*, 2024.
- [10] GitHub, “CodeQL: GitHub’s industry-leading semantic code analysis engine,” <https://codeql.github.com/>, 2024.
- [11] r2c, “Semgrep: Lightweight static analysis,” <https://semgrep.dev/>, 2024.
- [12] PyCQA, “Bandit: Security oriented static analyser for Python code,” <https://bandit.readthedocs.io/>, 2024.
- [13] H. Le *et al.*, “CodeRL: Mastering code generation through pretrained models and deep reinforcement learning,” in *NeurIPS*, 2022.
- [14] S. Dou *et al.*, “StepCoder: Improving code generation with reinforcement learning from compiler feedback,” in *ACL*, 2024.
- [15] S. S. Sijwali and S. Saha, “Scheduled partial-credit RL for reliable code generation with small language models (WIP),” in *Proceedings of the 27th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, ser. LCTES ’26. ACM, 2026.
- [16] Q.-A. Dang and C. Ngo, “Reinforcement learning for reasoning in small LLMs: What works and what doesn’t,” *arXiv preprint arXiv:2503.16219*, 2025.
- [17] S. Lu, N. Duan, H. Han, D. Guo, S.-w. Hwang, and A. Svyatkovskiy, “ReACC: A retrieval-augmented code completion framework,” in *ACL*, 2022.
- [18] Z. Z. Wang *et al.*, “CodeRAG-Bench: Can retrieval augment code generation?” in *Findings of the Association for Computational Linguistics: NAACL*, 2025.
- [19] X. Du *et al.*, “Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG,” *ACM Transactions on Software Engineering and Methodology*, 2026, Just Accepted.
- [20] G. Bhandari, A. Naseer, and L. Moonen, “CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software,” in *PROMISE*, 2021.
- [21] Y. Chen *et al.*, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” in *RAID*, 2023.
- [22] NIST SARD, “Juliet test suite v1.3 for C/C++,” <https://samate.nist.gov/SARD/test-suites/>, 2017.
- [23] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “BAAI/bge-base-en-v1.5: General-purpose English embedding model,” <https://huggingface.co/BAAI/bge-base-en-v1.5>, 2023.
- [24] D. Marjamaki and contributors, “Cppcheck: A static analysis tool for C/C++ code,” <https://cppcheck.sourceforge.io/>, 2024.
- [25] M. Bhatt *et al.*, “CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models,” *arXiv preprint arXiv:2404.13161*, 2024.
- [26] Y. Yang *et al.*, “SecCodePLT: A unified platform for evaluating the security of code GenAI,” *arXiv preprint arXiv:2410.11096*, 2024.
- [27] R. A. Dubniczky *et al.*, “CASTLE: Benchmarking dataset for static code analyzers and LLMs towards CWE detection,” in *International Symposium on Theoretical Aspects of Software Engineering (TASE)*, 2025.
- [28] M. L. Siddiq and J. C. Santos, “SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques,” in *MSR4P&S*, 2022.
- [29] J. Peng *et al.*, “CWEval: Outcome-driven evaluation on functionality and security of LLM code generation,” in *IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025.
- [30] B. Hui *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [31] A. Ahmadian *et al.*, “Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs,” in *ACL*, 2024.
- [32] H. Dong *et al.*, “RAFT: Reward rAnked fineTuning for generative foundation model alignment,” *Transactions on Machine Learning Research*, 2023.
- [33] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *ICLR*, 2022.
- [34] A. Lozhkov *et al.*, “StarCoder 2 and The Stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024.
- [35] D. Guo *et al.*, “UniXcoder: Unified cross-modal pre-training for code representation,” in *ACL*, 2022.
- [36] Z. Feng *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP*, 2020.
- [37] Y. Ding *et al.*, “Vulnerability detection with code language models: How far are we?” in *ICSE*, 2025.
- [38] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.